



Université de Tunis - El Manar
Ecole Nationale d'Ingénieurs de Tunis



Calcul de la valeur efficace d'un signal sinusoïdal injecté sur canal₁

**Réalisé par : Hamza MAMI
Maram TURKI**

Encadré par : Mr.Khaled JLASSI

Année universitaire:2025-2026

OBJECTIF

L'objectif de ce programme est de générer un signal sinusoïdal analogique à l'aide du DAC du microcontrôleur STM32F4, avec une amplitude et une fréquence réglables par des potentiomètres, puis de mesurer et calculer en temps réel la valeur efficace (RMS) de ce signal à l'aide de l'ADC.

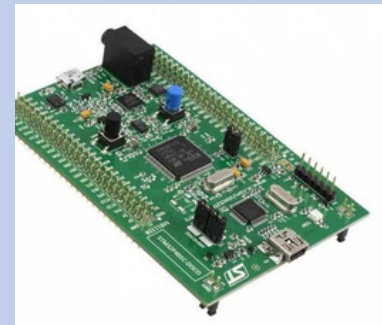
Le programme utilise :

- **une table de sinus pour produire un signal périodique stable,**
- **le SysTick pour assurer un échantillonnage temporel précis,**
- **le DAC pour la génération du signal analogique,**
- **l'ADC pour l'acquisition du signal de retour,**
- **le calcul de la valeur efficace du signal sinusoïdal par la méthode de la somme des carrés, après suppression de l'offset continu, sur un nombre fixe d'échantillons.**

OUTILS DE TRAVAIL

Microcontrôleur STM32F407VG

LE STM32F407VG EST LE MICROCONTRÔLEUR PRINCIPAL DU SYSTÈME. IL PERMET LA GÉNÉRATION DU SIGNAL SINUSOÏDAL À L'AIDE DU DAC, SON ACQUISITION VIA L'ADC, AINSI QUE LE TRAITEMENT NUMÉRIQUE NÉCESSAIRE AU CALCUL DE LA VALEUR EFFICACE (RMS).



Keil μ Vision

Keil μ Vision est l'environnement de développement utilisé pour écrire, compiler et déboguer le programme en langage C, puis programmer le microcontrôleur STM32.



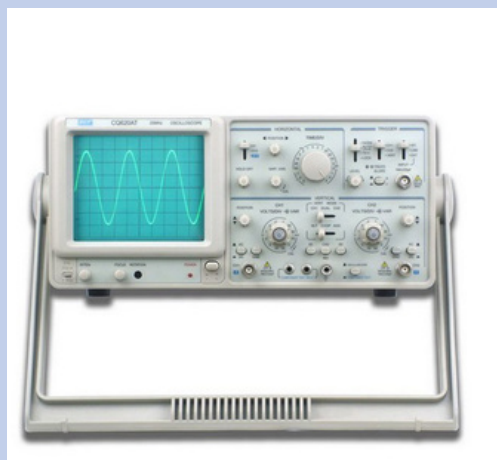
Potentiomètres

Les potentiomètres sont utilisés comme interfaces de réglage. Ils permettent de modifier en temps réel l'amplitude et la fréquence du signal sinusoïdal en fournissant une tension analogique variable au STM32.



Oscilloscope

L'oscilloscope sert à visualiser le signal sinusoïdal généré. Il permet de vérifier la forme, l'amplitude et la fréquence du signal, et de valider expérimentalement le bon fonctionnement du système.



MÉTHODOLOGIE DU TRAVAIL

- 1- Configuration des périphérique d'entrées-sorties GPIO**
- 2- Configuration du convertisseur analogique numérique DAC**
- 3- Configuration du convertisseur numérique analogique ADC**
- 4- Configuration du TIMER SysTick**
- 5- Génération du signal sinusoïdal**
- 6- Calcul de la valeur efficace**

CONFIGURATION DES PÉRIPHÉRIQUES

```
#include "stm32f4xx.h"
#include "stm32f4xx_rcc.h"

void config(void)
{
    /* Activer horloges GPIO et DAC/ADC */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA |
        RCC_AHB1Periph_GPIOC, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
}
```

La fonction config() active les horloges des différents périphériques du microcontrôleur STM32F407VG afin de les préparer à l'utilisation. Les horloges des ports GPIO A et C sont activées pour permettre la configuration des broches en entrée ou sortie analogique. L'horloge du DAC est activée pour générer des signaux analogiques sur PA4, tandis que celle de l'ADC1 est activée pour permettre la lecture des tensions provenant des potentiomètres ou du retour du DAC. Sans cette étape, les périphériques ne pourraient pas fonctionner correctement.

GÉNÉRATION DU SIGNAL SINUSOÏDAL PAR LE DAC

1- Initialisation de la table de sinus

```
/* ===== Table sinus ===== */  
void init_sin_table(void)  
{  
    for (int i = 0; i < NPOINTS; i++)  
    {  
        sin_table[i] = (uint16_t)(  
            (sinf(PI2 * i / NPOINTS) + 1.0f) * (DAC_MAX / 2.0  
            f)  
        );  
    }  
}
```

La table de sinus contient 256 valeurs représentant une période complète du signal. Chaque valeur est calculée avec la fonction sinus, décalée pour rester positive et adaptée à l'échelle du DAC (0 à 4095). Le DAC lit ensuite ces valeurs point par point pour générer un signal sinusoïdal analogique stable et contrôlable

2- Génération du signal via le DAC

Les valeurs de la table sinus sont envoyées au DAC point par point à chaque interruption SysTick. Chaque point est centré et ajusté selon l'amplitude désirée. En parcourant rapidement toute la table et en recommençant à zéro, le DAC produit un signal sinusoïdal analogique continu et stable. Tell que :

$$f_{signal} = \frac{F_s}{NPOINTS \times \text{diviseur_freq}}$$

$$V_{out} = \text{valeur_table} \times \frac{ADC_{pot}}{4095}$$

```
/* ===== Generation sinus ===== */
freq_cnt++;
if (freq_cnt >= diviseur_freq)
{
    freq_cnt = 0;

    // Sinus normalise [-1 ; +1]
    float sin_norm =
        ((float)sin_table[index_sin] / (DAC_MAX / 2.0f)) -
        1.0f;

    dbg_sin_norm = sin_norm;    // pour Logic Analyzer

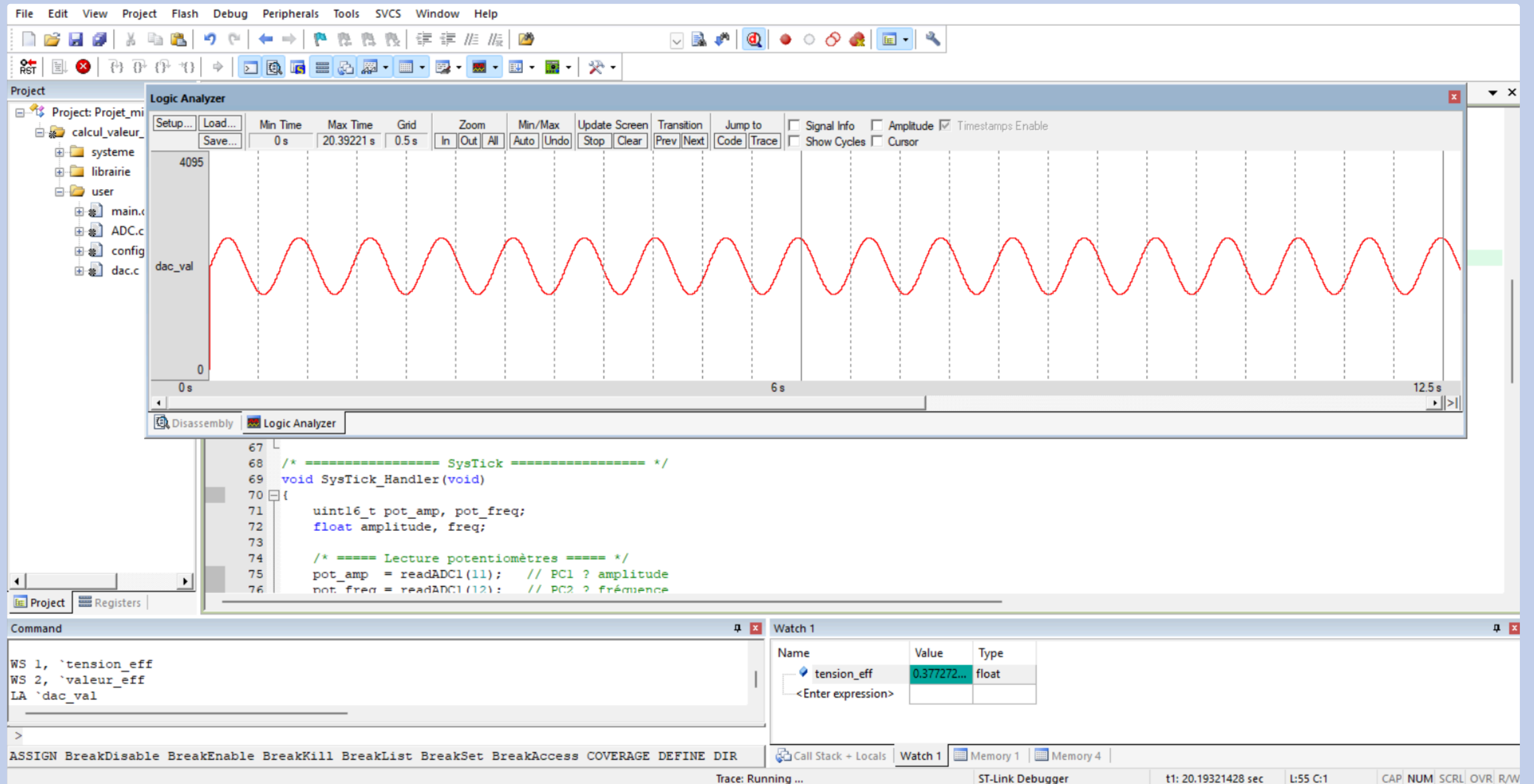
    // Application amplitude + offset DAC
    float dac_float =
        (DAC_MAX / 2.0f) + (DAC_MAX / 2.0f) * amplitude *
        sin_norm;

    if (dac_float < 0.0f) dac_float = 0.0f;
    if (dac_float > DAC_MAX) dac_float = DAC_MAX;

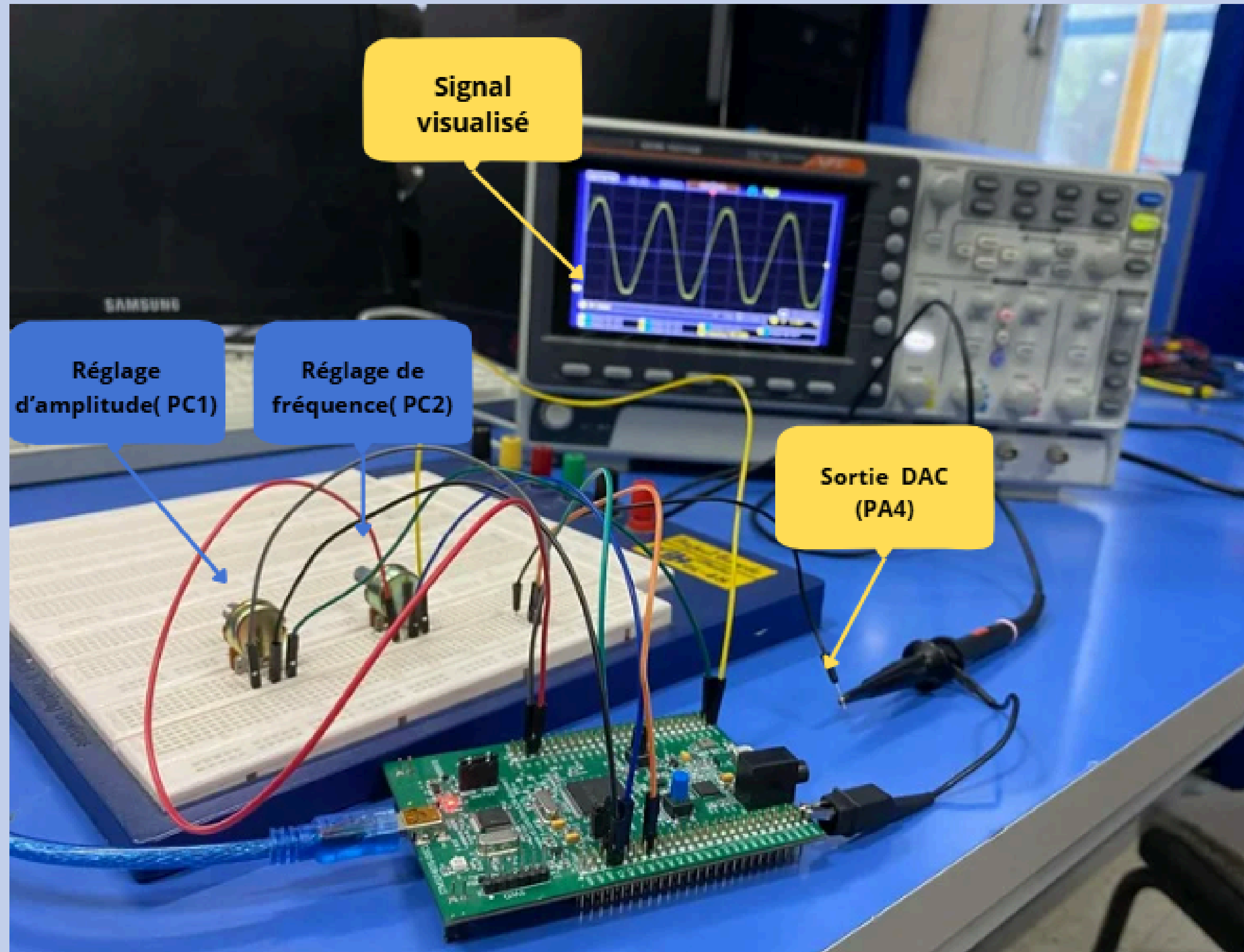
    dac_val = (uint16_t)dac_float;
    DAC_SetChannel1Data(DAC_Align_12b_R, dac_val);

    index_sin++;
    if (index_sin >= NPOINTS) index_sin = 0;
}
```

3- visualisation du signal sur logic analyser



4- Validation expérimentale du signal DAC



. Modélisation Théorique

.Paramètres de Configuration

- Fréquence d'échantillonnage (FS) : 20 kHz (soit 20000 interruptions par seconde).
- Résolution du signal (NPOINTS) : 256 points (taille de la table de sinus en mémoire).

La fréquence physique réelle ($f_{réelle}$) observée correspond au cas où le signal parcourt l'intégralité de la table sans division supplémentaire (vitesse maximale d'avancement) :

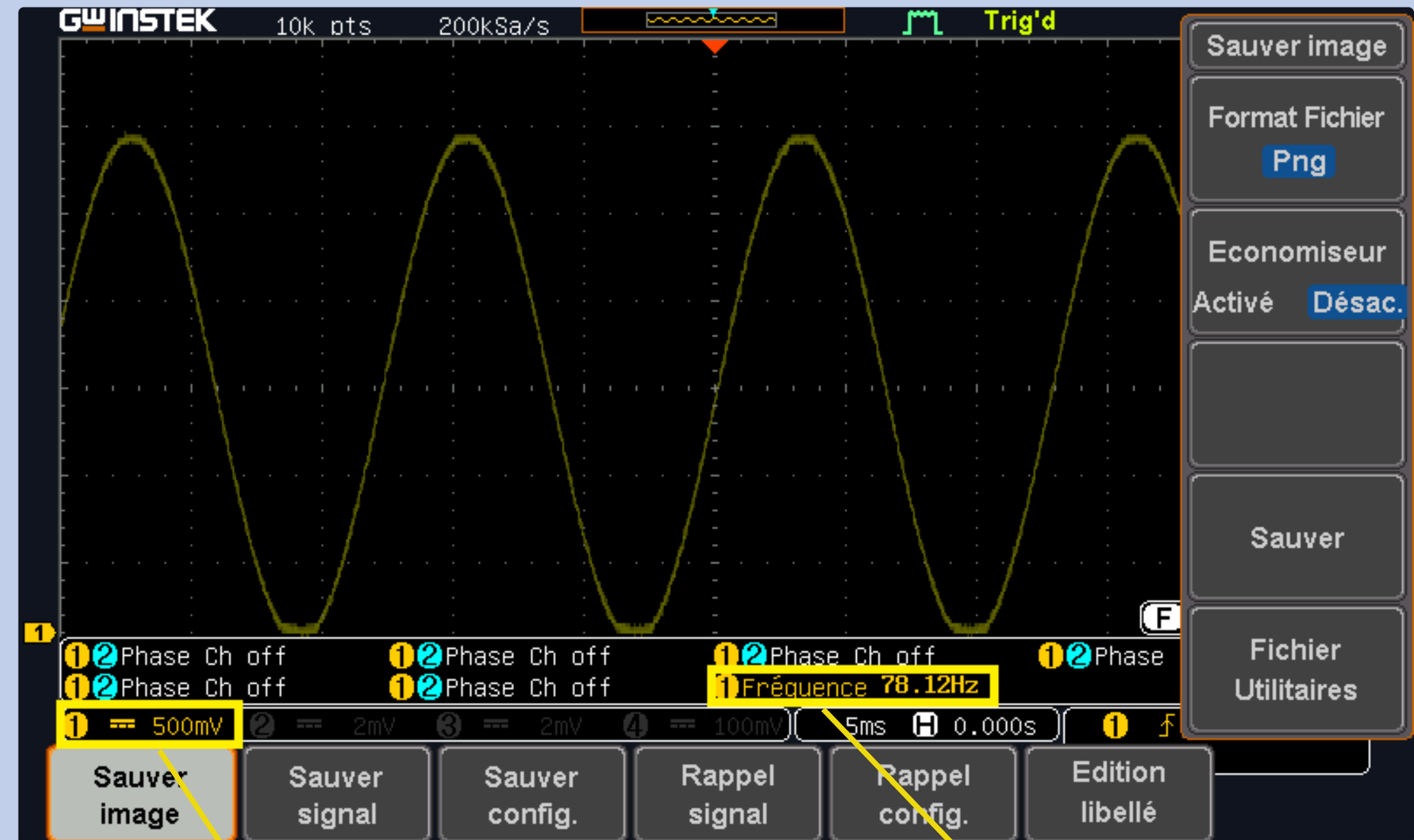
$$f_{réelle} = FS / \{NPOINTS\}$$

Calcul :

$$f_{réelle} = 20000 / 256 = 78,125 \text{ Hz}$$

Conclusion : La valeur mesurée de 78,12 Hz valide la précision du Timer et confirme que le signal est reconstitué point par point à partir de la fréquence d'échantillonnage programmée.

.Observation : Le signal est une sinusoïde parfaite



Amplitude :
Proche de 3 V

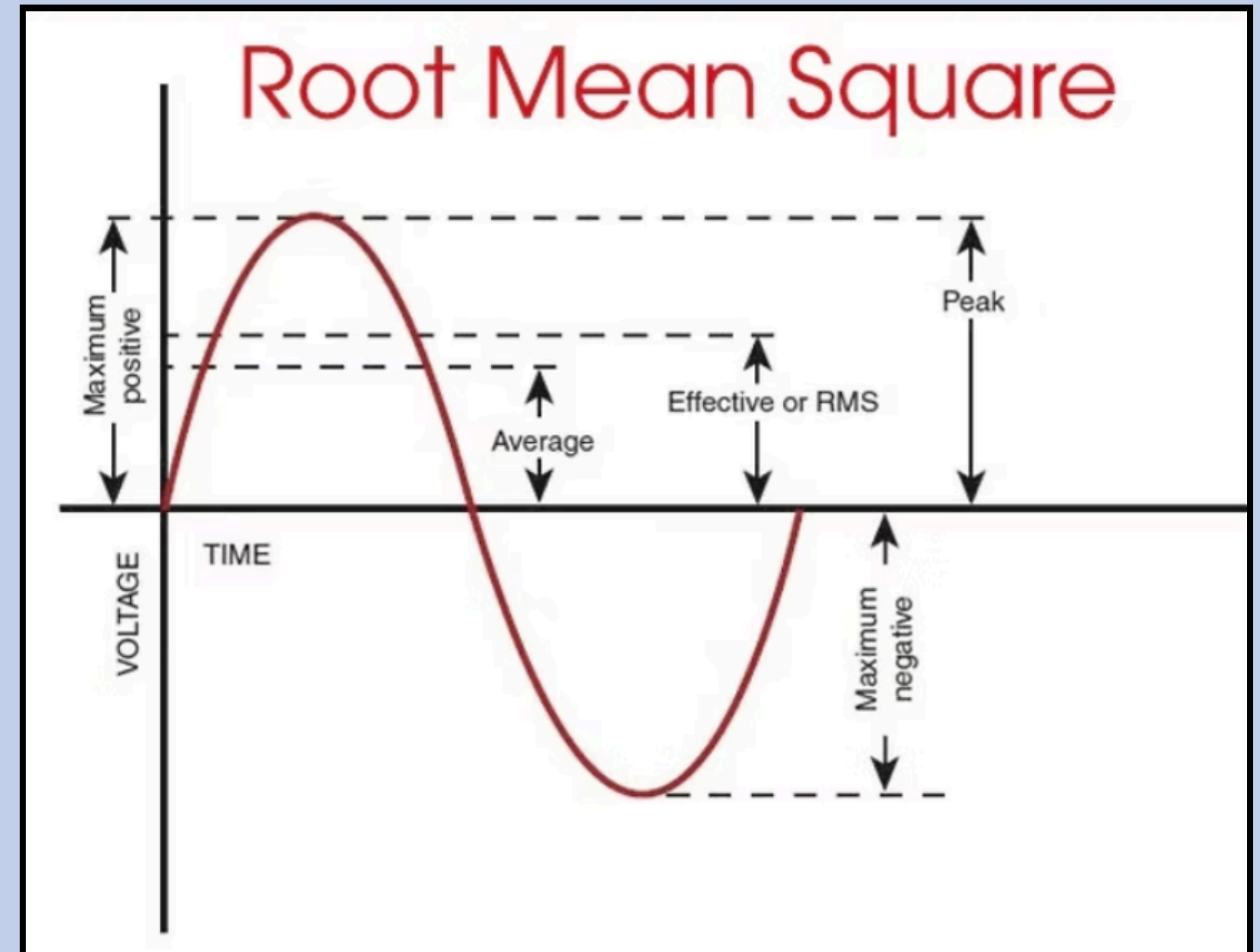
Fréquence mesurée :
78,12 Hz, ce qui
confirme exactement
la précision de notre
algorithme et du DAC.

CALCUL DE LA VALEUR EFFICACE

1- Principe:

- Le RMS (Root Mean Square) correspond à la valeur efficace d'un signal sinusoïdal. Elle correspond à la racine carrée de la moyenne des carrés des valeurs instantanées du signal.

$$V_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{i=1}^N V_i^2}$$



2- Fonctionnement du code:

1. Acquisition & Conversion : Lecture de la sortie DAC sur la pin PA4 et conversion en tension réelle.

2. Suppression de l'Offset : Recentrage du signal sur 0V en soustrayant $V_{REF} / 2$.

3. Intégration : Accumulation du carré des échantillons sur une période de N points.

4. Résultat Final : Calcul de la racine carrée de la moyenne pour obtenir V_{eff} .

```
/* ===== Calcul RMS (methode somme des carres) ===== */  
  
adc_val = readADC1(4);    // PA4 : retour DAC  
  
// Conversion ADC -> tension  
float v_adc = ((float)adc_val * VREF) / 4095.0f;  
  
// Suppression offset 1.65 V  
float v_ac = v_adc - (VREF / 2.0f);  
  
// Somme des carres  
s += v_ac * v_ac;  
rms_cnt++;  
  
if (rms_cnt >= NPOINTS)  
{  
    tension_eff = sqrtf(s / rms_cnt);  
    s = 0.0f;  
    rms_cnt = 0;  
}
```